Route Constraints in ASP.NET Core WEB API

What is Route Constraint?

A **Route Constraint** is a validation rule applied to route parameters. It ensures that incoming URL segments match specific **data types**, **ranges**, or **patterns** BEFORE the request reaches any controller action.

Syntax: {parameterName:constraint}

Example: {id:int} → only accepts integers (rejects abc)

Analogy: Like a bouncer at a club – checks our ID and only lets us in meet the rules.

Why Use Route Constraints?

Benefit	Explanation
Early Validation	Invalid requests rejected at routing level, before controller executes
Route Disambiguation	Distinguishes between similar routes (e.g., {id:int} vs {name:alpha})
Better Performance	Invalid requests don't waste resources reaching controllers
Cleaner Code	No need for validation logic inside action methods

The Problem: Without Constraints (Ambiguous Routes)

When two actions have similar route patterns, ASP.NET Core doesn't know which one to call.

```
[ApiController]
[Route("api/[controller]")]
public class EmployeeController : ControllerBase
{
    // BOTH routes match ANY value - AMBIGUOUS!
    [HttpGet("{EmployeeId}")]
    public string GetEmployeeDetails(int EmployeeId) { ... }

    [HttpGet("{EmployeeName}")]
    public string GetEmployeeDetails(string EmployeeName) { ... }
}
```

Problem: /api/employee/10 matches BOTH methods. Result: AmbiguousMatchException

Solution: Add constraints to tell them apart.

Built-in Route Constraints

1: Type Constraint (:int, :bool, :datetime, :decimal, :guid)

Restricts parameter to a specific data type.

Constraint	Valid Example	Invalid Example
{id:int}	/employee/5	/employee/abc
{id:bool}	/employee/true	/employee/123
{date:datetime}	/employee/2024-01-15	/employee/hello

```
[HttpGet("{EmployeeId:int}")]  // Only integers reach here
public string GetEmployeeDetails(int EmployeeId)
{
    return $"EmployeeId: {EmployeeId}";
}

[HttpGet("{EmployeeName}")]    // Strings reach here (default)
public string GetEmployeeDetails(string EmployeeName)
{
    return $"EmployeeName: {EmployeeName}";
}
```

URL	Matches
/api/employee/10	Integer version
/api/employee/smith	String version

2: Range Constraint (:range(min, max))

Restricts numeric values to a specific range (inclusive).

```
// Rating must be between 1 and 5
[HttpGet("rating/{rating:int:range(1,5)}")]
public IActionResult GetByRating(int rating)
{
    return Ok($"Rating: {rating}");  // Only 1-5 reach here
}
```

URL	Results
/api/product/rating/4	Matches (4 is between 1-5)
/api/product/rating/9	404 Not Found

3: Min/Max Constraint (:min(value), :max(value)

Sets minimum or maximum numeric limits.

```
// Price must be at least 100, at most 100000
[HttpGet("price/{minPrice:int:min(100)}/to/{maxPrice:int:max(100000)}")]
public IActionResult GetByPriceRange(int minPrice, int maxPrice)
{
    return Ok($"Price range: {minPrice} - {maxPrice}");
}
```

URL	Results
/api/product/price/5000/to/20000	Matches
/api/product/price/50/to/2000	404 Not Found (50 < min 100)

4: Alpha Constraint (:alpha)

Allows only alphabetic characters (A-Z, a-z). No numbers, no spaces, no symbols.

```
[HttpGet("category/{category:alpha}")]
public IActionResult GetByCategory(string category)
{
    return Ok($"Category: {category}");
}
```

URL	Results
/api/product/category/electronics	Matches
/api/product/category/electronics123	404 Not Found (contains numbers)

5: Length Constraints (:minlength(n), :maxlength(n), :length(n))

Control string length.

```
// Code must be between 6 and 8 characters
[HttpGet("code/{code:minlength(6):maxlength(8)}")]
public IActionResult GetByCode(string code)
{
    return Ok($"Code: {code}");
}

// Code must be exactly 7 characters
[HttpGet("exact/{code:length(7)}")]
public IActionResult GetByExactCode(string code)
{
    return Ok($"Code: {code}");
}
```

URL	Results
/api/product/code/PRD1234	Matches (7 chars)
/api/product/code/PRD12	404 Not Found (5 characters)

6: Regex Constraint (:regex(pattern))

Matches custom pattern using Regular Expression.

```
// SKU format: PROD- followed by exactly 4 digits
[HttpGet("sku/{sku:regex(^PROD-[[0-9]]{{4}}$)}")]
public IActionResult GetBySku(string sku)
{
    return Ok($"SKU: {sku}");
}
```

Pattern Explanation:

- **^** → Start of strng
- **PROD** → Literal text "PROD-"
- **[0-9]** → Any digit 0-9
- **{{4}}** → Exactly 4 digits
- **$** → End of string

URL	Results
/api/product/sku/PROD-1001	Matches
/api/product/sku/prod1001	404 Not Found (Wrong format)

Now we are going to set up the project,

Project Setup

<u>Create the project:</u>

Command: dotnet new webapi -n LibraryApi5 -f net8.0

<u>Navigate to the project:</u>

Command: cd LibraryApi5

Install Packages

Before installing the packages, we should make sure that we are in the project folder 'LibraryApi5' in terminal. If not, then navigate to the exact project folder in terminal.

Command 1: dotnet add package Microsoft.EntityFrameworkCore --version 8.0.0

Command 2: dotnet add package Microsoft.EntityFrameworkCore.SqlServer --version 8.0.0

Command 3: dotnet add package Microsoft.EntityFrameworkCore.Tools --version 8.0.0

Command 4: dotnet add package Microsoft.EntityFrameworkCore.Design --version 8.0.0

To check, whether the packages are successfully set up or not, we have to look over .csproj file,

LibraryApi5.csproj

```
<Project Sdk="Microsoft.NET.Sdk.Web">

  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
  </PropertyGroup>

  <ItemGroup>
    <PackageReference Include="Microsoft.AspNetCore.OpenApi" Version="8.0.24" />
    <PackageReference Include="Microsoft.EntityFrameworkCore" Version="8.0.0" />
    <PackageReference Include="Microsoft.EntityFrameworkCore.Design" Version="8.0.0">
      <IncludeAssets>runtime; build; native; contentfiles; analyzers; buildtransitive
      </IncludeAssets>
      <PrivateAssets>all</PrivateAssets>
    </PackageReference>
    <PackageReference Include="Microsoft.EntityFrameworkCore.SqlServer" Version="8.0.0" />
    <PackageReference Include="Microsoft.EntityFrameworkCore.Tools" Version="8.0.0">
      <IncludeAssets>runtime; build; native; contentfiles; analyzers; buildtransitive
      </IncludeAssets>
      <PrivateAssets>all</PrivateAssets>
    </PackageReference>
    <PackageReference Include="Swashbuckle.AspNetCore" Version="6.6.2" />
  </ItemGroup>

</Project>
```

Step 1: Create Model

Path: LibraryApi5/Models/Product.cs

Book.cs

Models > Book.cs > ...

```
namespace LibraryApi5.Models
{
    0 references
    public class Book
    {
        0 references
        public int Id { get; set; }
        0 references
        public string Title { get; set; } = string.Empty;
        0 references
        public string Author { get; set; } = string.Empty;
        0 references
        public string Genre { get; set; } = string.Empty; // Fantasy, Mystery, etc.
        0 references
        public int Rating { get; set; }                   // 1 to 5 stars
        0 references
        public string ISBN { get; set; } = string.Empty;  // 978-0-547-92822-7 format
        0 references
        public int PublishedYear { get; set; }
        0 references
        public bool IsAvailable { get; set; }
        0 references
        public decimal Price { get; set; }
    }
}
```

Step 2: Create In-Memory Data (10 Books)

Path: LibraryApi5/Data/BookData.cs

BookData.cs

Data > BookData.cs > ...

```
using LibraryApi5.Models;

namespace LibraryApi5.Data
{
    0 references
    public static class BookData
    {
        0 references
        public static List<Book> Books = new()
        {
            new Book {
                Id = 1,
                Title = "The Hobbit",
                Author = "J.R.R. Tolkien",
                Genre = "Fantasy",
                Rating = 5,
                ISBN = "978-0-547-92822-7",
                PublishedYear = 1937,
                IsAvailable = true,
                Price = 15.99m
            },
            new Book {
                Id = 2,
                Title = "1984",
                Author = "George Orwell",
                Genre = "Dystopian",
                Rating = 4,
                ISBN = "978-0-452-28423-4",
                PublishedYear = 1949,
                IsAvailable = false,
                Price = 12.99m
            },
            new Book {
                Id = 3,
                Title = "Pride and Prejudice",
                Author = "Jane Austen",
                Genre = "Romance",
                Rating = 5,
                ISBN = "978-0-14-143951-8",
                PublishedYear = 1813,
                IsAvailable = true,
                Price = 9.99m
            },
            new Book {
                Id = 4,
                Title = "The Great Gatsby",
                Author = "F. Scott Fitzgerald",
                Genre = "Classic",
                Rating = 4,
                ISBN = "978-0-7432-7356-5",
                PublishedYear = 1925,
                IsAvailable = true,
                Price = 11.99m
            },
            new Book {
                Id = 5,
                Title = "Harry Potter",
                Author = "J.K. Rowling",
                Genre = "Fantasy",
                Rating = 5,
                ISBN = "978-0-439-70818-2",
                PublishedYear = 1997,
                IsAvailable = true,
                Price = 19.99m
            },
            new Book {
                Id = 6,
                Title = "Moby Dick",
                Author = "Herman Melville",
                Genre = "Adventure",
                Rating = 3,
                ISBN = "978-0-14-243724-7",
                PublishedYear = 1851,
                IsAvailable = false,
                Price = 14.99m
            },
            new Book {
                Id = 7,
                Title = "The Shining",
                Author = "Stephen King",
                Genre = "Horror",
                Rating = 4,
                ISBN = "978-0-385-12167-5",
                PublishedYear = 1977,
                IsAvailable = true,
                Price = 13.99m
            },
            new Book {
                Id = 8,
                Title = "Dune",
                Author = "Frank Herbert",
                Genre = "Sci-Fi",
                Rating = 5,
                ISBN = "978-0-441-17271-9",
                PublishedYear = 1965,
                IsAvailable = true,
                Price = 17.99m
            },
            new Book {
                Id = 9,
                Title = "The Alchemist",
                Author = "Paulo Coelho",
                Genre = "Fiction",
                Rating = 4,
                ISBN = "978-0-06-250217-4",
                PublishedYear = 1988,
                IsAvailable = true,
                Price = 10.99m
            },
```

```
            new Book {
                Id = 10,
                Title = "It Ends With Us",
                Author = "Colleen Hoover",
                Genre = "Romance",
                Rating = 4,
                ISBN = "978-1-501-11336-5",
                PublishedYear = 2016,
                IsAvailable = true,
                Price = 12.99m
            }
        };
    }
}
```

Step 3: Program.cs (Minimal)

Path: LibraryApi5/Program.cs

Program.cs

```
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

var app = builder.Build();

if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();
app.UseAuthorization();
app.MapControllers();

app.Run();
```

Step 4: Controller WITH Route Constraints

Path: LibraryApi5/Controllers/BookController.cs

BookController.cs

Controllers > BookController.cs > ...

```
using LibraryApi5.Data;
using Microsoft.AspNetCore.Mvc;

namespace LibraryApi5.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    0 references
    public class BookController : ControllerBase
    {
        // ==================================================
        // TYPE CONSTRAINT  :int - Only numeric IDs  allowed
        // URL: GET /api/book/5
        // Invalid: /api/book/abc --> 404
        // ==================================================
        [HttpGet("{id:int}")]
        0 references
        public IActionResult GetById(int id)
        {
            var book = BookData.Books.FirstOrDefault(b => b.Id == id);
            return book is not null
                    ? Ok(book)
                    : NotFound($"No book found with ID: {id}");
        }

        // ==================================================
        // RANGE CONSTRAINT  - Ratting must be between 1 aned 5
        // URL: GET /api/book/rating/5
        // Invalid: /api/book/rating/9 --> 404
        // ==================================================
        [HttpGet("rating/{rating:int:range(1, 5)}")]
        0 references
        public IActionResult GetByRating(int rating)
        {
            var books = BookData.Books.Where(b => b.Rating == rating)
                        .ToList();
            return books.Any()
                    ? Ok(books)
                    : NotFound($"No books found with rating: {rating}");
        }

        // ==================================================
        // ALPHA CONSTRAINT - Only letters allowed in genre
        // URL: GET /api/book/genre/fantasy
        // Invalid: /api/book/genre/fantasy123 --> 404
        // ==================================================
        [HttpGet("genre/{genre:alpha}")]
        0 references
        public IActionResult GetByGenre(string genre)
        {
            var books = BookData.Books.Where(b => b.Genre
                        .Equals(genre, StringComparison.OrdinalIgnoreCase)
                        ).ToList();
            return books.Any()
                    ? Ok(books)
                    : NotFound($"No books found in genre: {genre}");
        }

```

```
        // ======================================================
        // LENGTH CONSTRAINT - ISBN must be exactly 17 characters
        // URL: GET /api/book/isbn/980-0-547-92822-7
        // ======================================================
        [HttpGet("isbn/{isbn:length(17)}")]
        0 references
        public IActionResult GetByIsbn(string isbn)
        {
            var book = BookData.Books.FirstOrDefault(b => b.ISBN == isbn);
            return book is not null
                ? Ok(book)
                : NotFound($"No book found with ISBN: {isbn}");
        }

        // =======================================================
        // MINLENGTH & MAXLENGTH - Title between 3 & 50 characters
        // URL: GET /api/book/title/The Hobbit
        // =======================================================
        [HttpGet("title/{title:minlength(3):maxlength(50)}")]
        0 references
        public IActionResult GetByTitle(string title)
        {
            var book = BookData.Books.FirstOrDefault(b => b.Title.Equals(
                title, StringComparison.OrdinalIgnoreCase
            ));
            return book is not null
                ? Ok(book)
                : NotFound($"No book found with title: {title}");
        }

```

```
        // ========================================================
        // REGIX CONSTRAINT - Author name (letters, spaces, dots
        // allowed)
        // URL: GET /api/book/author/Tolkien
        // ========================================================
        [HttpGet("author/{author:regex(^[[A-Za-z. ]]+$)}")]
        0 references
        public IActionResult GetByAuthor(string author)
        {
            var books = BookData.Books.Where(b => b.Author
                .Contains(author,
                StringComparison.OrdinalIgnoreCase
            )).ToList();
            return books.Any()
                ? Ok(books)
                : NotFound($"No books found by author: {author}");
        }

```

```
        // ========================================================
        // MIN & MAX CONSTRAINT - Price range ($5 to $100)
        // URL: GET /api/book/price/10/to/20
        // ========================================================
        [HttpGet
        ("price/{minPrice:decimal:min(5)}/to/{maxPrice:decimal:max(100)}")
        0 references
        public IActionResult GetByPriceRange
        (decimal minPrice, decimal maxPrice)
        {
            if(minPrice > maxPrice)
            {
                return
                BadRequest("Minimum price cannot exceed maximum price");
            }
            var books = BookData.Books
                .Where(b => b.Price >= minPrice && b.Price <= maxPrice)
                .ToList();
            return books.Any()
                ? Ok(books)
    : NotFound($"No books found between ${minPrice} and ${maxPrice}");
        }

        // ========================================================
        // RANGE CONSTRAINT - Year must be between 1800 and 2025
        // URL: GET /api/book/year/1937
        // ========================================================
        [HttpGet("year/{year:int:range(1800,2025)}")]
        0 references
        public IActionResult GetByYear(int year)
        {
            var books = BookData.Books
                .Where(b => b.PublishedYear == year).ToList();
            return books.Any()
                ? Ok(books)
                : NotFound($"No books found published in {year}");
        }

        // ========================================================
        // MULTIPLE CONSTRAINTS - Genre + Rating combination
        // URL: GET /api/book/filter/fantasy/5
        // ========================================================
        [HttpGet("filter/{genre:alpha}/{rating:int:range(1,5)}")]
        0 references
        public IActionResult GetByGenreAndRating(string genre, int rating)
        {
            var books = BookData.Books
                .Where(b => b.Genre.Equals(genre,
                StringComparison.OrdinalIgnoreCase)
                && b.Rating == rating)
                .ToList();
            return books.Any()
                ? Ok(books)
                : NotFound($"No {genre} books found with rating {rating}");
        }
    }
}
```

Custom Route Constraint

Scenario: Allow only specific genres (Fantasy, Mystery, Romance, Sci-Fi)

<u>Step 1: Create LibraryApi5/Constraints/AllowedGenresConstraint.cs</u>

AllowedGenresConstraint.cs

Constraints > AllowedGenresConstraint.cs > ...

```
namespace LibraryApi5.Constraints
{
    0 references
    public class AllowedGenresConstraint : IRouteConstraint
    {
        1 reference
        private static readonly string[] AllowedGenres =
        {
            "fantasy",
            "mystery",
            "romance",
            "scifi",
            "dystopian",
            "classic",
            "horror",
            "adventure"
        };

        0 references
        public bool Match(HttpContext? httpContext,
                          IRouter? route,
                          string parameterName,
                          RouteValueDictionary values,
                          RouteDirection routeDirection
        )
        {
            if (!values.TryGetValue(parameterName, out var rawValue))
            {
                return false;
            }

            var genre = rawValue?.ToString();

            return !string.IsNullOrEmpty(genre) &&
                   Array.Exists(AllowedGenres, g =>
                       g.Equals(genre, StringComparison.OrdinalIgnoreCase)
                   );
        }
    }
}
```

Step 2: Register in Program.cs

Program.cs

Program.cs

```
using LibraryApi5.Constraints;

var builder = WebApplication.CreateBuilder(args);

// Service Registration Section ---
// Add our custom route constraint here
builder.Services.Configure<RouteOptions>(options =>
{
    options.ConstraintMap.Add("allowedGenres",
        typeof(AllowedGenresConstraint));
});

builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

var app = builder.Build();

if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();
app.UseAuthorization();
app.MapControllers();

app.Run();
```

Step 3: Use in Controller (BookController.cs)

```
153        // ====================================================
154        // CUSTOM CONSTRAINT - Only letters allowed some specific genres
155        // URL: GET /api/book/strict-genre/fantasy
156        // Invalid: /api/book/strict-genre/toys --> 404 (not allowed in list)
157        // ====================================================
158        [HttpGet("strict-genre/{genre:allowedGenres}")]
           0 references
159        public IActionResult GetByStrictGenre(string genre)
160        {
161            var books = BookData.Books
162                .Where(b => b.Genre.Equals(genre, StringComparison.OrdinalIgnoreCase))
163                .ToList();
164            return Ok(books);
165        }
```

Test our API – All Endpoints with Expected Responses

Let's build and run the application. And here we will use the command line or cli. We have to navigate to the exact project folder (LibraryApi5) and run those following commands,

Build the application,

Command: *dotnet build*

```
PS E:\PracticeProjects\DotnetApi\Routing\LibraryApi5> dotnet build
  Determining projects to restore...
  All projects are up-to-date for restore.
  LibraryApi5 -> E:\PracticeProjects\DotnetApi\Routing\LibraryApi5\bin\Debug\n
  et8.0\LibraryApi5.dll

Build succeeded.
    0 Warning(s)
    0 Error(s)

Time Elapsed 00:00:03.61
```

To run the application,

Command: *dotnet run* (we usually get the URL from here to see application response).

We can run the project and can directly navigate to application response or Swagger UI.

Use can run the command,

Command: *dotnet watch run*

```
PS E:\PracticeProjects\DotnetApi\Routing\LibraryApi5> dotnet watch run
dotnet watch 🔥 Hot reload enabled. For a list of supported edits, see https://
aka.ms/dotnet/hot-reload.
   💡 Press "Ctrl + R" to restart.
dotnet watch 🔧 Building...
  Determining projects to restore...
  All projects are up-to-date for restore.
  LibraryApi5 -> E:\PracticeProjects\DotnetApi\Routing\LibraryApi5\bin\Debug\n
  et8.0\LibraryApi5.dll
dotnet watch 🚀 Started
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: http://localhost:5025
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
      Hosting environment: Development
info: Microsoft.Hosting.Lifetime[0]
      Content root path: E:\PracticeProjects\DotnetApi\Routing\LibraryApi5
warn: Microsoft.AspNetCore.HttpsPolicy.HttpsRedirectionMiddleware[3]
      Failed to determine the https port for redirect.
```

Book

GET /api/Book/{id}

GET /api/Book/rating/{rating}

GET /api/Book/genre/{genre}

GET /api/Book/isbn/{isbn}

GET /api/Book/title/{title}

GET /api/Book/author/{author}

GET /api/Book/price/{minPrice}/to/{maxPrice}

GET /api/Book/year/{year}

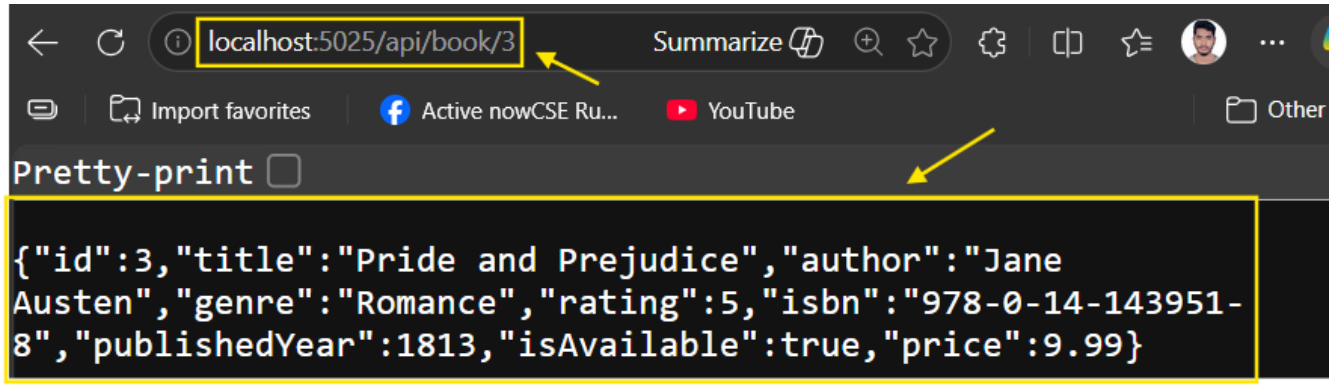
GET /api/Book/filter/{genre}/{rating}

GET /api/Book/strict-genre/{genre}

Test 1: Type Constraint – Valid Integer

Request: GET http:localhost:5025/api/book/3

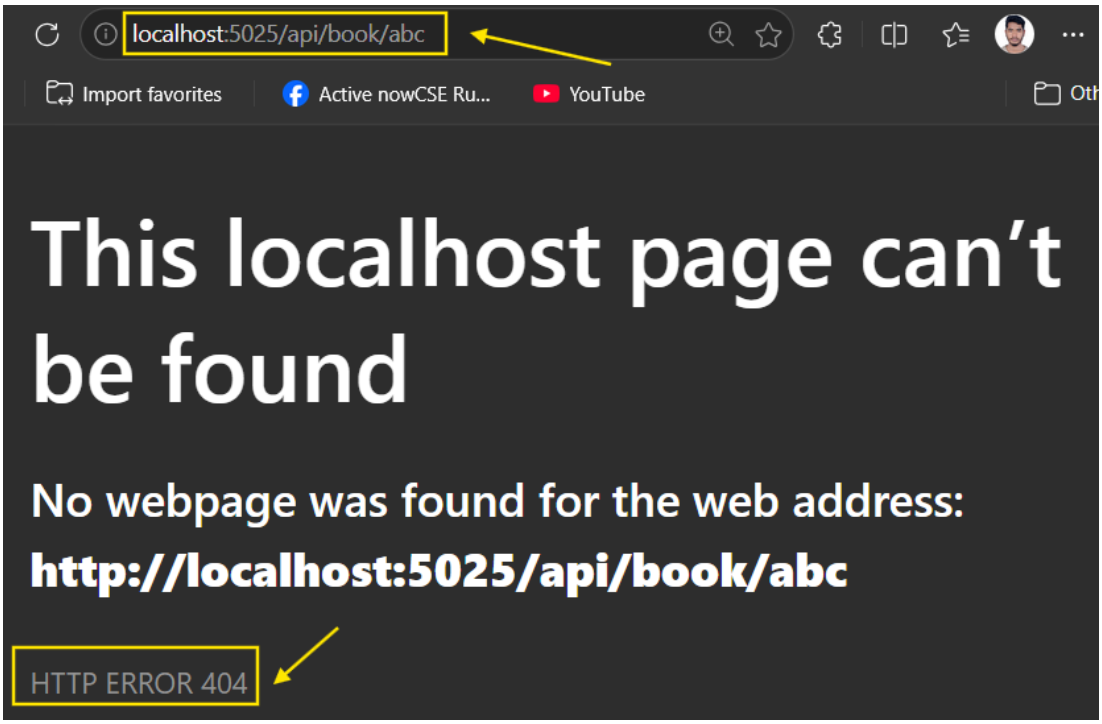
Response:



Test 2: Type Constraint - Invalid (Non-Integer)

Request: GET http:localhost:5025/api/book/abc

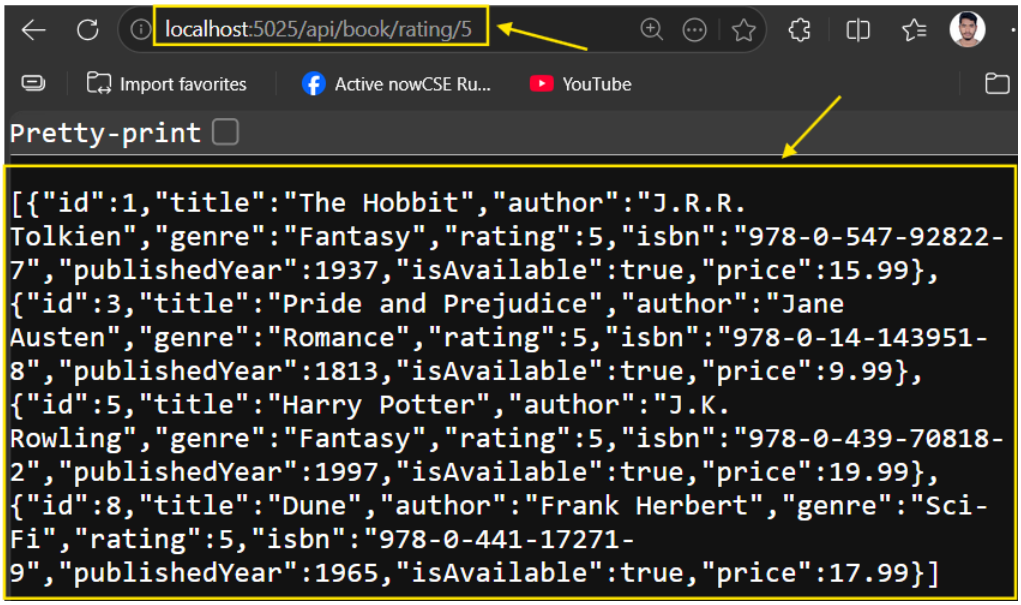
Response:



Test 3: Range Constraint - Valid Rating

Request: GET http:localhost:5025/api/book/rating/5

Response:

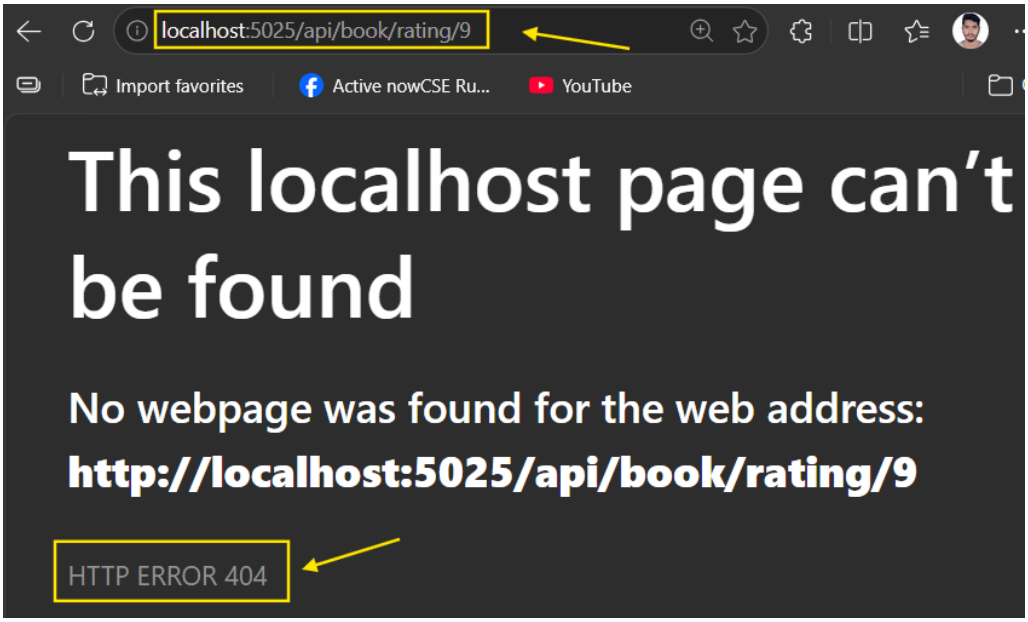



```
[{"id":1,"title":"The Hobbit","author":"J.R.R.
Tolkien","genre":"Fantasy","rating":5,"isbn":"978-0-547-92822-
7","publishedYear":1937,"isAvailable":true,"price":15.99},
{"id":3,"title":"Pride and Prejudice","author":"Jane
Austen","genre":"Romance","rating":5,"isbn":"978-0-14-143951-
8","publishedYear":1813,"isAvailable":true,"price":9.99},
{"id":5,"title":"Harry Potter","author":"J.K.
Rowling","genre":"Fantasy","rating":5,"isbn":"978-0-439-70818-
2","publishedYear":1997,"isAvailable":true,"price":19.99},
{"id":8,"title":"Dune","author":"Frank Herbert","genre":"Sci-
Fi","rating":5,"isbn":"978-0-441-17271-
9","publishedYear":1965,"isAvailable":true,"price":17.99}]
```


Test 4: Range Constraint - Invalid Rating

Request: GET http:localhost:5025/api/book/rating/9

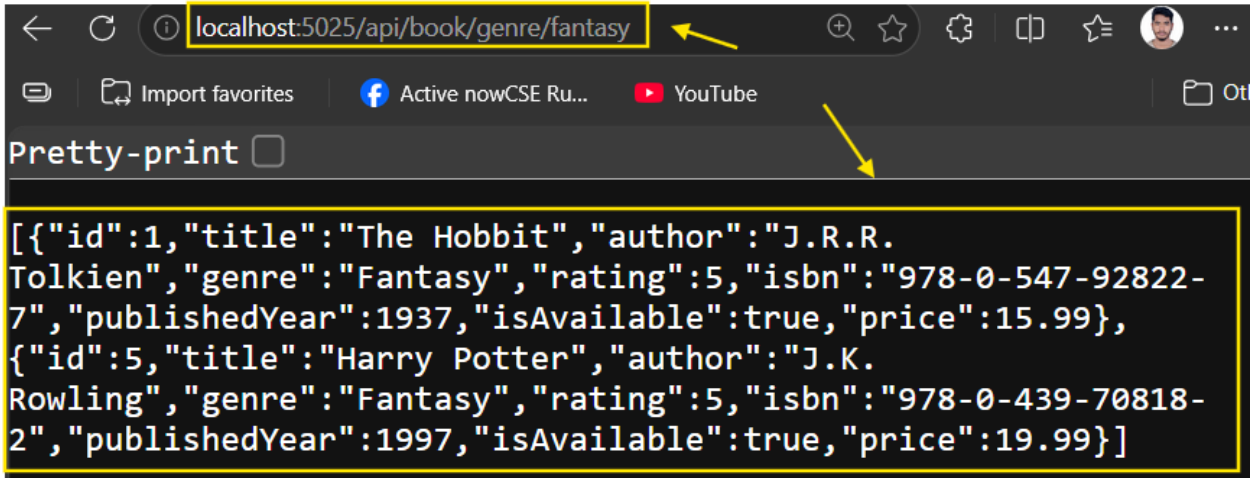
Response:



Test 5: Alpha Constraint - Valid Genre

Request: GET http:localhost:5025/api/book/genre/fantasy

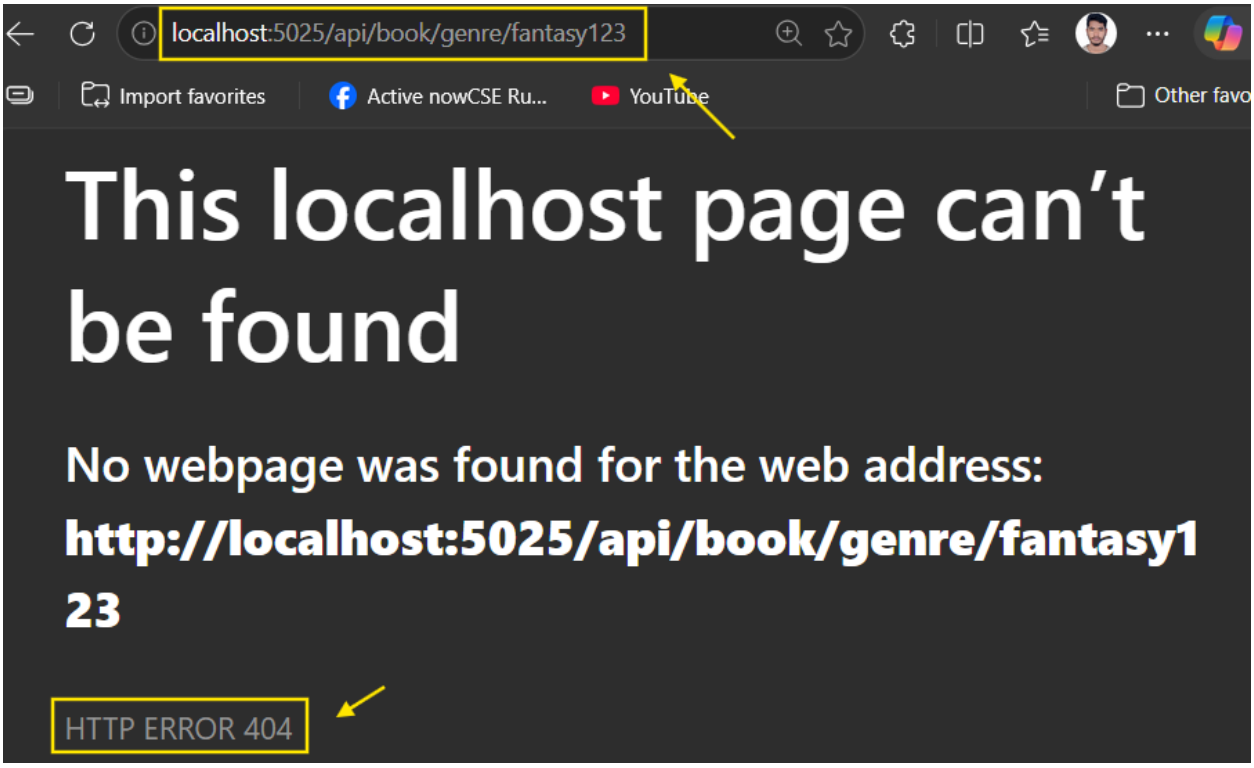
Response:



Test 6: Alpha Constraint - Invalid (Contains Numbers)

Request: GET http:localhost:5025/api/book/genre/fantasy123

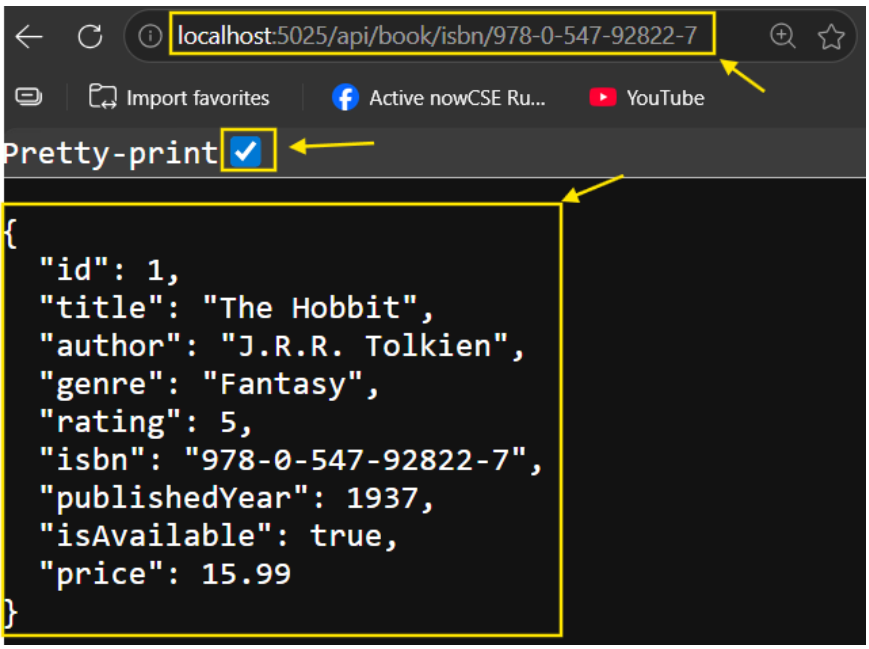
Response:



Test 7: Length Constraint - Valid ISBN

Request: GET http:localhost:5025/api/book/isbn/978-0-547-92822-7

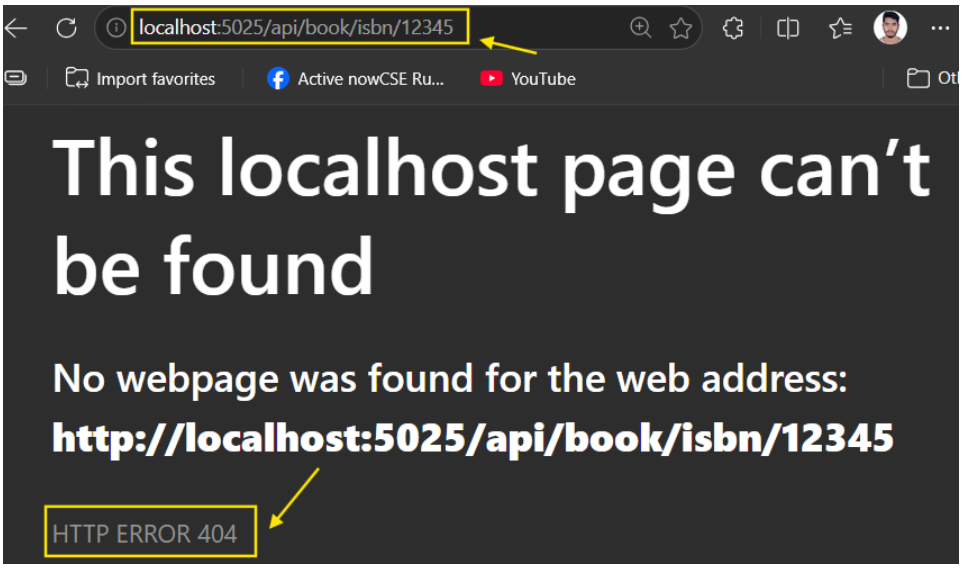
Response:



Test 8: Length Constraint - Invalid Length of ISBN

Request: GET http:localhost:5025/api/book/isbn/12345

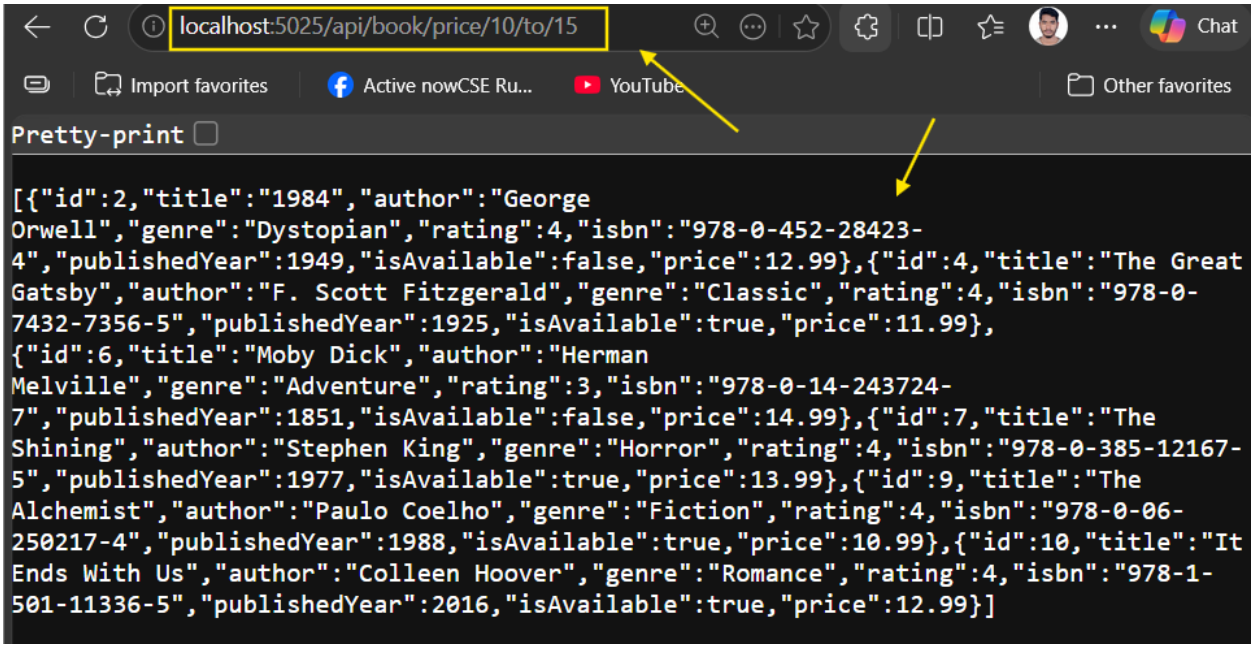
Response:



Test 9: Min/Max Price Constraint

Request: GET http:localhost:5025/api/book/price/10/to/15

Response:

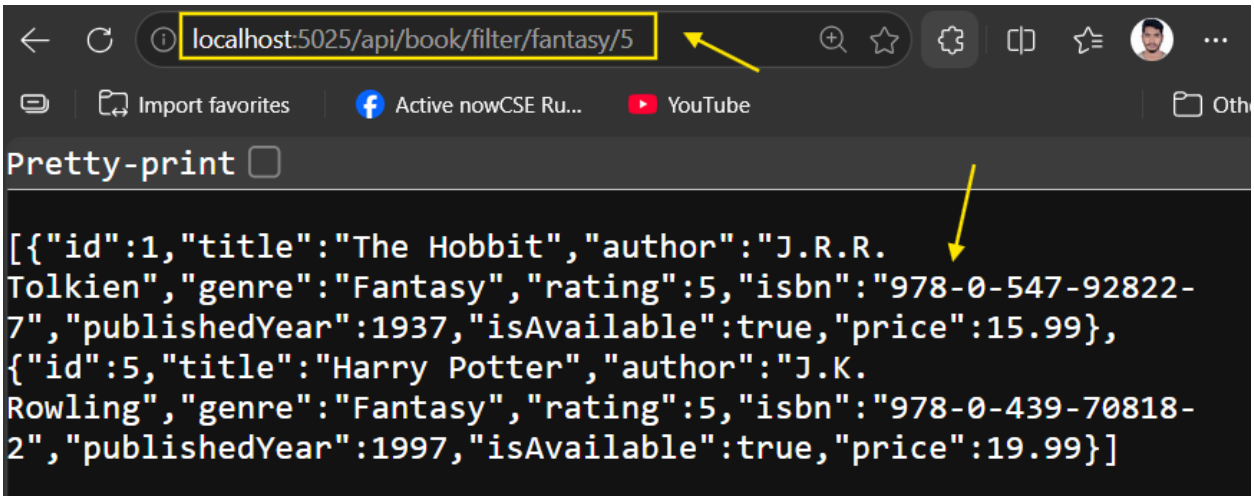


[{"id":2,"title":"1984","author":"George Orwell","genre":"Dystopian","rating":4,"isbn":"978-0-452-28423-4","publishedYear":1949,"isAvailable":false,"price":12.99},{"id":4,"title":"The Great Gatsby","author":"F. Scott Fitzgerald","genre":"Classic","rating":4,"isbn":"978-0-7432-7356-5","publishedYear":1925,"isAvailable":true,"price":11.99},{"id":6,"title":"Moby Dick","author":"Herman Melville","genre":"Adventure","rating":3,"isbn":"978-0-14-243724-7","publishedYear":1851,"isAvailable":false,"price":14.99},{"id":7,"title":"The Shining","author":"Stephen King","genre":"Horror","rating":4,"isbn":"978-0-385-12167-5","publishedYear":1977,"isAvailable":true,"price":13.99},{"id":9,"title":"The Alchemist","author":"Paulo Coelho","genre":"Fiction","rating":4,"isbn":"978-0-06-250217-4","publishedYear":1988,"isAvailable":true,"price":10.99},{"id":10,"title":"It Ends With Us","author":"Colleen Hoover","genre":"Romance","rating":4,"isbn":"978-1-501-11336-5","publishedYear":2016,"isAvailable":true,"price":12.99}]

Test 10: Multiple Constraints - Genre + Rating

Request: GET http:localhost:5025/api/book/filter/fantasy/5

Response:



[{"id":1,"title":"The Hobbit","author":"J.R.R. Tolkien","genre":"Fantasy","rating":5,"isbn":"978-0-547-92822-7","publishedYear":1937,"isAvailable":true,"price":15.99},{"id":5,"title":"Harry Potter","author":"J.K. Rowling","genre":"Fantasy","rating":5,"isbn":"978-0-439-70818-2","publishedYear":1997,"isAvailable":true,"price":19.99}]

Key Takeaways

Concept	Explanation
Route Constraints	Validation rules on route parameters using {param:constraint}
Purpose	Early validation, route disambiguation, better performance
Type Constraints	:int, :bool, :datetime, :decimal, :guid
Range Constraints	:range(min,max), :min(value), :max(value)
String Constraints	:alpha, :length(n), :minlength(n), :maxlength(n)
Regex Constraint	:regex(pattern) for custom patterns
Custom Constraints	Implement IRouteConstraint for domain-specific rules
Cannot Apply to Query Strings	Route constraints only work on URL path parameters